claude-windows / 54d4074a-5a8d-4058-b42d-a757c338132c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra\54d4074a-5a8d-4058-b42d-a757c338132c\subagents\workflows\wf_1c9ea2fd-eb8\agent-a84892d84db2cca19.jsonl`
- SHA-256: `d702757ef26a663c32c374ea1d97a3c36848670d7dc6fc4fb048fc5e63ec20b3`
- Source modified: `2026-06-21T19:34:35+00:00`
- Imported at: `2026-07-08T15:59:32+00:00`
- project: `wf_1c9ea2fd-eb8`
- session_id: `54d4074a-5a8d-4058-b42d-a757c338132c`

Transcript

1. user (2026-06-21T19:32:10.689Z)

Completeness critic for this PR (under C:/Users/avidu/Projects/khelsutra-cov). The task was: "ensure coverage is improved and not regressed" for a repo that previously had NO coverage measurement. The PR adds coverage tooling + a CI floor + regression tests (SelectionFooter render test raising the previously-uncovered floor-warning block 66%?~100%, and catalog-invariants locking the #77 audit rules). Measured coverage went 90.44?92.61 lines.
Assess what's MISSING or could be materially better:
1. Is the coverage IMPROVEMENT real and attributable, or mostly from excluding main.tsx/types.ts (i.e. did we actually add meaningful test coverage vs just reframing the denominator)? Be honest.
2. Are there cheap, high-value coverage wins left on the table (e.g. QueryBar.tsx funcs 40%, i18n/index.ts branch 25% on the syncDocumentLang fallback)? Worth this PR or a follow-up?
3. Does anything claim more than it delivers (e.g. comments overstating the guarantee)?
Return structured findings, prioritising the single most valuable addition if any.

2. assistant (2026-06-21T19:32:12.925Z)

I'll analyze this PR for completeness. Let me start by understanding the repo state and the changes.

3. user (2026-06-21T19:32:14.304Z)

9f1ec9c fix(i18n): native-review pass on the kn/hi SPA console catalogs (#77)
9a2d7d2 feat(site): A7 ? camera-placement diagram on the capture page (#76)
aa8d5e9 feat(site): A6 ? plain-language Request-Access GPU/storage questions (#75)
1cd84af feat(site): A3 ? localize recording-education + marketing to kn/hi, Noto on the site (#74)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process (analytics) (#73)
1b1fa7b feat(web+site): B3 ? trilingual marketing site via shared t()/data-i18n shim (D-CATALOG-HOME) (#70)
f7e2bd5 chore(site): add robots.txt + a "crawling / bot protection" runbook for khelsutra.guru (#71)
bea8a2a docs(appliance): flip T11 retention sweep to ? ? tool shipped (engine #299), scheduling tracked (#301) (#69)
46e0244 feat(web): A9 ? localized tap-only filter controls (#68)
a2292ef feat(web): MULTILINGUAL Phase A ? Spanish/Danish/Indonesian SPA catalogs (#67)
ca619fb feat(web): T4 ? in-UI file upload (the runnable-tool keystone) (#66)
4627830 docs(deploy): alpha deploy kit ? make Studio a shareable page (split: Pages + api.* tunnel + CF Access) (#65)
8bbc557 feat(web): VITE_API_BASE ? let the SPA target a split api.* origin (default same-origin /api) (#64)
92a6c45 feat(web): A8 ? background-processing reassurance + icon+text status chips (#63)
501c014 feat(web): A5 ? friendly engine-error mapping across build + load (shared errors.*) (#62)

```
---STATUS---
On branch chore/web-coverage-floor
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .github/workflows/test.yml
        modified:   web/package-lock.json
        modified:   web/package.json
        modified:   web/vite.config.ts

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        web/src/components/SelectionFooter.test.tsx
        web/src/i18n/catalog-invariants.test.ts

no changes added to commit (use "git add" and/or "git commit -a")
```

4. assistant (2026-06-21T19:32:18.673Z)

Let me look at the actual diff and the new test files.

5. user (2026-06-21T19:32:20.285Z)

```
diff --git a/.github/workflows/test.yml b/.github/workflows/test.yml
index b0b4d1d..afb2e19 100644
--- a/.github/workflows/test.yml
+++ b/.github/workflows/test.yml
@@ -29,7 +29,9 @@ jobs:
         run: for f in scripts/*.sh; do echo "checking $f"; bash -n "$f"; done

   web:
-    # Operator-console SPA (web/): typecheck + unit tests + production build gate the merge.
+    # Operator-console SPA (web/): typecheck + unit tests (with a coverage floor) + production
build
+    # gate the merge. `test:coverage` fails if coverage drops below the thresholds in
vite.config.ts,
+    # so a merge can't silently regress test coverage.
     runs-on: ubuntu-latest
     defaults:
       run:
@@ -43,5 +45,5 @@ jobs:
           cache-dependency-path: web/package-lock.json
       - run: npm ci
       - run: npm run typecheck
-      - run: npm run test
+      - run: npm run test:coverage
       - run: npm run build
diff --git a/web/package.json b/web/package.json
index 41ba748..b09de4f 100644
--- a/web/package.json
+++ b/web/package.json
@@ -9,7 +9,8 @@
     "build": "vite build",
     "preview": "vite preview",
     "typecheck": "tsc --noEmit",
-    "test": "vitest run"
+    "test": "vitest run",
+    "test:coverage": "vitest run --coverage"
   },
   "dependencies": {
     "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
@@ -29,6 +30,7 @@
     "@types/react": "^19.0.0",
```

```
     "@types/react-dom": "^19.0.0",
     "@vitejs/plugin-react": "^4.3.4",
+    "@vitest/coverage-v8": "^3.2.6",
     "jsdom": "^29.1.1",
     "tailwindcss": "^4.0.0",
     "typescript": "^5.7.0",
diff --git a/web/vite.config.ts b/web/vite.config.ts
index 6c9214b..0b4e471 100644
--- a/web/vite.config.ts
+++ b/web/vite.config.ts
@@ -20,5 +20,32 @@ export default defineConfig({
     include: ["src/**/*.test.{ts,tsx}"],
     // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files no-op.
     setupFiles: ["src/test/setup.ts"],
+    // Coverage floor (CI runs `npm run test:coverage`): a merge can't DROP coverage below
these
+    // thresholds, so untested new code is caught. `all` counts every source file (not only the
ones a
+    // test imported), so deleting a test surfaces here too. Excludes are non-logic files: the
app
+    // entry/bootstrap, type-only declarations, and the test harness itself.
+    coverage: {
+      provider: "v8",
+      reporter: ["text", "text-summary"],
+      all: true,
+      include: ["src/**/*.{ts,tsx}"],
+      exclude: [
+        "src/**/*.test.{ts,tsx}", // the tests
+        "src/test/**", // jsdom localStorage shim (test harness, not shipped logic)
+        "src/main.tsx", // app entry: mounts <App/> + bundles fonts ? nothing to unit-test
+        "src/api/types.ts", // type-only declarations (no runtime code)
+        "**/*.d.ts",
+      ],
+      // Floor set just below current (stmts/lines 92.6, branches 87.7, funcs 82.4) ? a real
ratchet
+      // that fails CI on a meaningful drop, with a little slack so it doesn't flap. Ratchet
upward as
+      // coverage grows. (functions is kept looser: App.tsx is the engine-orchestrating shell,
costly
+      // to exercise fully in unit tests.)
+      thresholds: {
+        statements: 92,
+        branches: 87,
+        functions: 80,
+        lines: 92,
+      },
+    },
   },
 });
```

6. user (2026-06-21T19:32:23.208Z)

```
1       // @vitest-environment jsdom
2       //
3       // Coverage + REGRESSION guard for SelectionFooter's floor-warning block (previously
uncovered) AND
4       // the kn/hi native-review fixes (#77). The warning renders footer.floorOne / floorAll /
5       // floorSomeLead+floorSomeRest ? the very keys the adversarial audit cleaned of the
leaked
6       // `min_shot_count` token and the "engine"/"server" jargon. Rendering the REAL catalog
in en/kn/hi and
7       // asserting the cleaned copy means a regression that re-introduces the token/jargon
fails right here
8       // (not just in the data-level catalog-invariants test) ? i.e. the bug can't reach the
screen.
```

```
9       import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
10      import { render, screen, cleanup, fireEvent } from "@testing-library/react";
11      import i18n from "../i18n";
12      import { SelectionFooter } from "./SelectionFooter";
13      import type { RallySegment } from "../api/types";
14
15      // Only the array LENGTH matters to the footer (floorSomeLead shows atRisk.length);
content is unused.
16      const risks = (n: number): RallySegment[] => Array.from({ length: n }, () => ({}) as
RallySegment);
17      const baseProps = { totalSec: 90, reelName: "", onReelNameChange: () => {}, onBuild: ()
=> {} };
18      const warnText = () => screen.getByTestId("floor-warning").textContent ?? "";
19      const buildBtn = () => screen.getByRole("button") as HTMLButtonElement; // the footer
has exactly one button
20
21      afterEach(cleanup);
22
23      describe("SelectionFooter ? floor warning renders the audit-clean copy (en)", () => {
24        beforeEach(async () => {
25          window.localStorage.clear();
26          await i18n.changeLanguage("en");
27        });
28
29        it("allDropped + a single selection ? floorOne, passive phrasing (no 'engine'), Build
blocked", () => {
30          render(<SelectionFooter {...baseProps} count={1} warning={{ minShotCount: 6, atRisk:
risks(1), allDropped: true }} />);
31          expect(warnText()).toContain("6"); // {min}
32          expect(warnText()).toMatch(/skipped/i);
33          expect(warnText()).not.toMatch(/engine/i); // audit: en is engine-free, so kn/hi/en
stay engine-free
34          expect(buildBtn().disabled).toBe(true); // every selected rally below the floor ?
build would fail
35        });
36
37        it("allDropped + many ? floorAll (count + min), Build blocked", () => {
38          render(<SelectionFooter {...baseProps} count={3} warning={{ minShotCount: 6, atRisk:
risks(3), allDropped: true }} />);
39          expect(warnText()).toContain("3"); // {count}
40          expect(warnText()).toContain("6"); // {min}
41          expect(warnText()).not.toMatch(/engine/i);
42          expect(buildBtn().disabled).toBe(true);
43        });
44
45        it("partial risk ? floorSomeLead + floorSomeRest, NO leaked min_shot_count token,
Build allowed", () => {
46          render(<SelectionFooter {...baseProps} count={5} warning={{ minShotCount: 6, atRisk:
risks(2), allDropped: false }} />);
47          const t = warnText();
48          expect(t).toContain("2"); // floorSomeLead count = atRisk.length
49          expect(t).toContain("6"); // {min}
50          expect(t).not.toMatch(/min_shot_count/); // audit: the internal code token must
never surface
51          expect(t).not.toMatch(/engine/i);
52          expect(buildBtn().disabled).toBe(false); // only SOME are at risk ? build can
proceed
53        });
54      });
55
56      describe("SelectionFooter ? floor warning is jargon-free in kn + hi (#77 regression)",
() => {
57        beforeEach(() => window.localStorage.clear());
58        afterEach(async () => {
59          await i18n.changeLanguage("en");
```

```
      });

      it("kn: floorOne + floorSome carry no '??????' and no leaked token", async () => {
        await i18n.changeLanguage("kn");
        render(<SelectionFooter {...baseProps} count={1} warning={{ minShotCount: 6, atRisk:
risks(1), allDropped: true }} />);
        expect(warnText()).not.toContain("??????");
        cleanup();
        render(<SelectionFooter {...baseProps} count={5} warning={{ minShotCount: 6, atRisk:
risks(2), allDropped: false }} />);
        expect(warnText()).not.toContain("??????");
        expect(warnText()).not.toMatch(/min_shot_count/);
      });

      it("hi: floorOne + floorSome carry no '????' and no leaked token", async () => {
        await i18n.changeLanguage("hi");
        render(<SelectionFooter {...baseProps} count={1} warning={{ minShotCount: 6, atRisk:
risks(1), allDropped: true }} />);
        expect(warnText()).not.toContain("????");
        cleanup();
        render(<SelectionFooter {...baseProps} count={5} warning={{ minShotCount: 6, atRisk:
risks(2), allDropped: false }} />);
        expect(warnText()).not.toContain("????");
        expect(warnText()).not.toMatch(/min_shot_count/);
      });
    });

    describe("SelectionFooter ? no-warning controls (summary, reel name, build button)", ()
=> {
      beforeEach(async () => {
        window.localStorage.clear();
        await i18n.changeLanguage("en");
      });

      it("no warning ? no floor-warning region; summary + enabled Build render", () => {
        render(<SelectionFooter {...baseProps} count={4} warning={null} />);
        expect(screen.queryByTestId("floor-warning")).toBeNull();
        expect(screen.getByTestId("selection-summary").textContent).toMatch(/4 rallies/);
        expect(buildBtn().disabled).toBe(false);
      });

      it("building ? the button shows the 'Building?' label and is disabled", () => {
        render(<SelectionFooter {...baseProps} count={2} warning={null} building />);
        expect(buildBtn().textContent).toMatch(/Building/);
        expect(buildBtn().disabled).toBe(true);
      });

      it("empty selection ? Build disabled", () => {
        render(<SelectionFooter {...baseProps} count={0} warning={null} />);
        expect(buildBtn().disabled).toBe(true);
      });

      it("editing the reel name and clicking Build fire their callbacks", () => {
        const onReelNameChange = vi.fn();
        const onBuild = vi.fn();
        render(<SelectionFooter {...baseProps} count={3} warning={null}
onReelNameChange={onReelNameChange} onBuild={onBuild} />);
        fireEvent.change(screen.getByLabelText("Reel name"), { target: { value: "finals" }
});
        expect(onReelNameChange).toHaveBeenCalledWith("finals");
        fireEvent.click(buildBtn());
        expect(onBuild).toHaveBeenCalled();
      });
    });

```

7. user (2026-06-21T19:32:23.575Z)

```
1       // Content regression guards for the localization catalogs ? the rules the adversarial
translation
2       // audit (#77) established, pinned so the specific bugs can never silently return. This
complements
3       // hygiene.test.ts (which proves key/placeholder PARITY) by asserting catalog CONTENT:
4       //   1. no internal code-token / <code> leaks in any locale,
5       //   2. the brand "KhelSutra" is never transliterated into an Indic script,
6       //   3. engine/server/bucket jargon appears ONLY on keys where the en source uses it,
7       //   4. the specific flagged mistranslations stay fixed.
8       // These are pure data assertions over repo-root /locales (D-CATALOG-HOME) ? no render
context needed.
9       import { describe, it, expect } from "vitest";
10      import { readFileSync, readdirSync } from "node:fs";
11      import { join } from "node:path";
12
13      const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");
14
15      type Flat = Record<string, string>;
16      function flatEntries(obj: Record<string, unknown>, prefix = ""): Flat {
17        const out: Flat = {};
18        for (const [k, v] of Object.entries(obj)) {
19          const key = prefix ? `${prefix}.${k}` : k;
20          if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
21          else out[key] = String(v);
22        }
23        return out;
24      }
25      const load = (loc: string): Flat =>
26        flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR, loc, "common.json"), "utf8")) as
Record<string, unknown>);
27
28      const ALL_LOCALES = readdirSync(LOCALES_DIR, { withFileTypes: true })
29        .filter((d) => d.isDirectory())
30        .map((d) => d.name);
31      const en = load("en");
32
33      describe("catalog content ? no internal token / HTML-tag leaks (every locale)", () => {
34        for (const loc of ALL_LOCALES) {
35          it(`${loc}: no 'min_shot_count' code token or <code> tag in any user-facing string`,
() => {
36            const offenders = Object.entries(load(loc))
37              .filter(([, v]) => /min_shot_count/.test(v) || /<\/?code>/i.test(v))
38              .map(([k]) => k);
39            expect(offenders).toEqual([]);
40          });
41        }
42      });
43
44      describe("catalog content ? the brand is never transliterated (Indic locales)", () => {
45        const TRANSLIT: Record<string, string> = { kn: "??????????", hi: "????????" };
46        for (const loc of ["kn", "hi"] as const) {
47          it(`${loc}: no Indic transliteration of "KhelSutra" anywhere`, () => {
48            const offenders = Object.entries(load(loc))
49              .filter(([, v]) => v.includes(TRANSLIT[loc]))
50              .map(([k]) => k);
51            expect(offenders).toEqual([]);
52          });
53          it(`${loc}: the ingest console keys keep the Latin brand`, () => {
54            const e = load(loc);
55            for (const key of ["ingest.intro", "ingest.reassure", "ingest.fileHint"]) {
56              expect(e[key], `${loc} ${key}`).toContain("KhelSutra");
57            }
```

```
58          });
59        }
60      });
61
62      describe("catalog content ? engine/server/bucket jargon only where en uses it (Indic
locales)", () => {
63        // en deliberately avoids these terms in user-facing body copy (it says "KhelSutra" /
"cloud-storage"
64        // / passive voice) ? EXCEPT the technical uriAriaLabel ("engine host", "bucket URL").
So the rule is
65        // self-maintaining: a localized jargon term is allowed ONLY on a key whose en value
also uses the
66        // English word. Reintroducing "engine"/"server"/"bucket" anywhere else fails here.
67        const JARGON: Record<string, ReadonlyArray<readonly [string, string]>> = {
68          kn: [
69            ["??????", "engine"],
70            ["??????", "server"],
71            ["?????", "bucket"],
72          ],
73          hi: [
74            ["????", "engine"],
75            ["?????", "server"],
76            ["????", "bucket"],
77          ],
78        };
79        for (const loc of ["kn", "hi"] as const) {
80          it(`${loc}: a jargon term appears only on keys whose en value also uses it`, () => {
81            const e = load(loc);
82            const violations: Array<{ key: string; term: string; en: string }> = [];
83            for (const [key, val] of Object.entries(e)) {
84              for (const [term, eng] of JARGON[loc]) {
85                if (val.includes(term) && !(en[key] ?? "").toLowerCase().includes(eng)) {
86                  violations.push({ key, term, en: en[key] ?? "(absent)" });
87                }
88              }
89            }
90            expect(violations).toEqual([]);
91          });
92        }
93      });
94
95      describe("catalog content ? flagged mistranslations stay fixed (Kannada/Hindi word
choice)", () => {
96        it("kn bulk.invert is not the 'rotate' mistranslation (????????)", () => {
97          expect(load("kn")["bulk.invert"]).not.toBe("????????");
98        });
99        it("kn result.dismiss is not the legalistic '?????????'", () => {
100         expect(load("kn")["result.dismiss"]).not.toBe("?????????");
101       });
102       it("hi result.dismiss is not the harsh '????? ????'", () => {
103         expect(load("hi")["result.dismiss"]).not.toBe("????? ????");
104       });
105     });
106
```

8. assistant (2026-06-21T19:32:26.115Z)

Now let me look at the source files to assess coverage. Let me run the coverage report to get the real numbers, and look at the key source files.

9. user (2026-06-21T19:32:27.659Z)

```
App.tsx
api
components
```

```
hooks
i18n
index.css
lib
main.tsx
test
vite-env.d.ts
---COMPONENTS---
CompileResult.tsx
IngestPanel.test.tsx
IngestPanel.tsx
LanguageSwitcher.tsx
QueryBar.examples.test.ts
QueryBar.tsx
RallyGrid.tsx
SelectionFooter.test.tsx
SelectionFooter.tsx
StatusChip.tsx
TapFilters.test.tsx
TapFilters.tsx
a4.touch.test.tsx
a8.status.test.tsx
build-error.test.tsx
upload.test.tsx
---I18N---
catalog-invariants.test.ts
hygiene.test.ts
i18n.test.tsx
index.ts
---API---
client.test.ts
client.ts
locale.test.ts
types.ts
upload.test.ts
```

10. user (2026-06-21T19:32:40.598Z)

```
{"ts":"2026-06-21T19:32:40.052Z","level":"info","event":"api.request","cid":"6a302b6e-
5db7-4e6f-a0ba-172b6b62d52a","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-
21T19:32:40.076Z","level":"error","event":"api.network_error","cid":"6a302b6e-5db7-4e6f-a0ba-
172b6b62d52a","method":"GET","url":"/api/config","error":"Error: no engine in test"}
{"ts":"2026-06-21T19:32:40.076Z","level":"warn","event":"config.unavailable","error":"ApiError:
Network error calling GET /api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-
21T19:32:40.076Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error
calling GET /api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-
21T19:32:40.076Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error
calling GET /api/config"}

 ? src/i18n/i18n.test.tsx (17 tests)
1467ms
   ? i18n scaffold (B1) > renders user-facing copy from the en
catalog (not raw keys)  645ms
```

```
 Test Files  22 passed (22)
      Tests  201 passed (201)
   Start at  01:02:33
   Duration  6.84s (transform 1.47s, setup 761ms, collect 9.65s, tests 6.93s,
environment 35.26s, prepare 6.70s)

 % Coverage report from v8
-------------------|---------|----------|---------|---------|-------------------
File               | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------|---------|----------|---------|---------|-------------------
All files          |   92.61 |    87.74 |   82.35 |   92.61 |
 src               |   74.89 |    78.04 |      25 |   74.89 |
  App.tsx          |   74.89 |    78.04 |      25 |   74.89 | ...10-215,242-244
 src/api           |    98.1 |    89.36 |     100 |    98.1 |
  client.ts        |    98.1 |    89.36 |     100 |    98.1 | 174-176
 src/components    |   96.21 |    89.13 |   72.72 |   96.21 |
  ...ileResult.tsx |     100 |      100 |     100 |     100 |
  IngestPanel.tsx  |   95.48 |     90.9 |   66.66 |   95.48 | 40-42,44-46
  ...eSwitcher.tsx |     100 |    83.33 |     100 |     100 | 27
  QueryBar.tsx     |   86.84 |    76.92 |      40 |   86.84 | ...41-43,68-70,82
  RallyGrid.tsx    |   98.11 |    73.33 |      50 |   98.11 | 42
  ...ionFooter.tsx |     100 |      100 |     100 |     100 |
  StatusChip.tsx   |     100 |      100 |     100 |     100 |
  TapFilters.tsx   |     100 |      100 |     100 |     100 |
 src/hooks         |   92.89 |    76.92 |     100 |   92.89 |
  useIngestJob.ts  |   92.52 |    78.57 |     100 |   92.52 | ...08-110,141-142
  ...ySelection.ts |     100 |    83.33 |     100 |     100 | 9
  useUpload.ts     |    91.8 |    70.58 |     100 |    91.8 | 76-78,82-83
 src/i18n          |     100 |       25 |     100 |     100 |
  index.ts         |     100 |       25 |     100 |     100 | 72-76
 src/lib           |   97.22 |    94.61 |     100 |   97.22 |
  errorText.ts     |     100 |      100 |     100 |     100 |
  errors.ts        |     100 |      100 |     100 |     100 |
  floor.ts         |     100 |      100 |     100 |     100 |
  format.ts        |     100 |      100 |     100 |     100 |
  log.ts           |   81.81 |     87.5 |     100 |   81.81 | 13-14
  query.ts         |   97.07 |    93.45 |     100 |   97.07 | ...95,284,286,288
  selection.ts     |     100 |     92.3 |     100 |     100 | 30
-------------------|---------|----------|---------|---------|-------------------

=============================== Coverage summary ===============================
Statements   : 92.61% ( 1266/1367 )
Branches     : 87.74% ( 365/416 )
Functions    : 82.35% ( 70/85 )
Lines        : 92.61% ( 1266/1367 )
================================================================================
```

11. assistant (2026-06-21T19:32:44.343Z)

Now I have the real coverage table. Let me verify the baseline claim (90.44 ? 92.61) by checking what coverage looks like on master without the new tests, and examine the specific uncovered areas the task flagged. Let me look at the SelectionFooter component and the i18n index fallback.

12. user (2026-06-21T19:32:45.210Z)

```
1       // i18n scaffold (tracker B1) ? the shared localization foundation for KhelSutra Studio.
2       //
3       // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4       //   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5       //   - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
the catalog,
6       //     not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
7       //   - one negotiation order shared with the marketing site (B3);
8       //   - always-fallback to `en` so a missing translation degrades to English, never a raw
```

```
key.
9       import i18n from "i18next";
10      import { initReactI18next } from "react-i18next";
11      import LanguageDetector from "i18next-browser-languagedetector";
12      import ICU from "i18next-icu";
13      // Catalogs are HOISTED to repo-root /locales (D-CATALOG-HOME) ? the single source of
truth shared
14      // by this SPA and the marketing site's t() shim (tracker B3). Vite resolves the
workspace root via
15      // the repo `.git` marker, so importing the parent-dir /locales works in dev, build, and
vitest.
16      import enCommon from "../../../locales/en/common.json";
17      import knCommon from "../../../locales/kn/common.json";
18      import hiCommon from "../../../locales/hi/common.json";
19      import esCommon from "../../../locales/es/common.json";
20      import daCommon from "../../../locales/da/common.json";
21      import idCommon from "../../../locales/id/common.json";
22
23      // Supported locales: en/kn/hi foundation (FRIENDLY D6) + MULTILINGUAL_EXPANSION Phase A
(es/da/id,
24      // Latin, catalog-only). Anything else falls back to `en`.
25      export const SUPPORTED_LOCALES = ["en", "kn", "hi", "es", "da", "id"] as const;
26      export type Locale = (typeof SUPPORTED_LOCALES)[number];
27      export const DEFAULT_LOCALE: Locale = "en";
28
29      // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
B3), so a
30      // visitor's language choice carries across both surfaces.
31      export const LANG_STORAGE_KEY = "khelsutra.lang";
32
33      // Catalogs live in repo-root /locales (D-CATALOG-HOME), consumed by both surfaces. `kn`
+ `hi`
34      // shipped (B5/B6, AI-drafted ? native review pending). A missing key in any locale
still falls back
35      // to `en` ? the by-design invariant.
36      const resources = {
37        en: { common: enCommon },
38        kn: { common: knCommon },
39        hi: { common: hiCommon },
40        es: { common: esCommon },
41        da: { common: daCommon },
42        id: { common: idCommon },
43      } as const;
44
45      void i18n
46        .use(ICU)
47        .use(LanguageDetector)
48        .use(initReactI18next)
49        .init({
50          resources,
51          fallbackLng: DEFAULT_LOCALE,
52          supportedLngs: [...SUPPORTED_LOCALES],
53          nonExplicitSupportedLngs: true, // en-IN -> en
54          defaultNS: "common",
55          ns: ["common"],
56          interpolation: { escapeValue: false }, // React already escapes
57          returnNull: false,
58          // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
59          detection: {
60            order: ["localStorage", "querystring", "navigator"],
61            lookupLocalStorage: LANG_STORAGE_KEY,
62            lookupQuerystring: "lang",
63            caches: ["localStorage"],
64          },
65          react: { useSuspense: false },
```

```
66        });
67
68      // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
69      // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
70      function syncDocumentLang(lng: string) {
71        if (typeof document !== "undefined") {
72          document.documentElement.lang = (i18n.resolvedLanguage || lng ||
DEFAULT_LOCALE).split("-")[0];
73        }
74      }
75      i18n.on("languageChanged", syncDocumentLang);
76      syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
77
78      export default i18n;
79
```

13. user (2026-06-21T19:32:45.644Z)

```
1       // Sticky compile footer (PER_RALLY_SELECTION.md P4): live selection summary, a reel-
name input,
2       // the Build button, and ? above all ? an HONEST min_shot_count warning. The engine
silently drops
3       // selected rallies below stitching.min_shot_count; we surface that BEFORE Build so a
rally never
4       // vanishes without a reason (D7). We warn, we do NOT override the floor (D11).
5       import { useTranslation, Trans } from "react-i18next";
6       import { mmss } from "../lib/format";
7       import type { FloorRisk } from "../lib/floor";
8
9       interface Props {
10        count: number;
11        totalSec: number;
12        reelName: string;
13        onReelNameChange: (v: string) => void;
14        onBuild: () => void;
15        /** A compile is in flight (Build wired in P5). */
16        building?: boolean;
17        /** Non-null when some selected rallies sit below the engine floor. */
18        warning: FloorRisk | null;
19      }
20
21      export function SelectionFooter({ count, totalSec, reelName, onReelNameChange, onBuild,
building, warning }: Props) {
22        const { t } = useTranslation();
23        const buildDisabled = count === 0 || building || warning?.allDropped === true;
24
25        return (
26          <footer className="fixed inset-x-0 bottom-0 border-t border-black/10 bg-white/95
backdrop-blur">
27            {warning && (
28              <div
29                role="status"
30                data-testid="floor-warning"
31                className={
32                  "border-b px-6 py-2 text-sm " +
33                  (warning.allDropped
34                    ? "border-red-200 bg-red-50 text-red-800"
35                    : "border-amber-200 bg-amber-50 text-amber-900")
36                }
37              >
38                {warning.allDropped ? (
39                  count === 1 ? (
40                    <Trans i18nKey="footer.floorOne" values={{ min: warning.minShotCount }}
```

```
components={{ b: <strong /> }} />
41                  ) : (
42                    <Trans i18nKey="footer.floorAll" values={{ count, min:
warning.minShotCount }} components={{ b: <strong /> }} />
43                  )
44                ) : (
45                  <>
46                    <strong>{t("footer.floorSomeLead", { count: warning.atRisk.length
})}</strong>{" "}
47                    <Trans i18nKey="footer.floorSomeRest" values={{ min: warning.minShotCount
}} components={{ code: <code /> }} />
48                  </>
49                )}
50              </div>
51            )}
52
53            <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-3 px-6 py-3">
54              <span className="text-sm text-ink/80" data-testid="selection-summary">
55                <strong className="text-ink">{t("footer.count", { count })}</strong>
{t("footer.summaryRest", { total: mmss(totalSec) })}
56              </span>
57
58              <label className="ml-auto flex items-center gap-2 text-sm text-ink/60">
59                <span className="hidden sm:inline">{t("footer.reelName")}</span>
60                <input
61                  type="text"
62                  value={reelName}
63                  onChange={(e) => onReelNameChange(e.target.value)}
64                  placeholder={t("footer.reelNamePlaceholder")}
65                  aria-label={t("footer.reelName")}
66                  className="min-h-11 w-40 rounded-lg border border-black/15 bg-white px-3
py-1.5 text-ink outline-none focus:border-green"
67                />
68              </label>
69
70              <button
71                type="button"
72                onClick={onBuild}
73                disabled={buildDisabled}
74                className="min-h-11 rounded-full bg-green px-5 py-2 text-sm font-semibold
text-[#04140D] disabled:opacity-40"
75              >
76                {building ? t("footer.building") : t("footer.build", { count })}
77              </button>
78            </div>
79          </footer>
80        );
81      }
82
```

14. assistant (2026-06-21T19:32:49.456Z)

Now let me verify the baseline claim. The task says coverage went 90.44 ? 92.61. Let me confirm by checking what the SelectionFooter floor block looked like before (66% claim) and reconstruct the baseline. Let me stash the new tests and re-run to get the true "before" number with the same exclude config.

15. user (2026-06-21T19:33:01.065Z)

```
removed new tests; running coverage with same vite.config exclude
File               | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------|---------|----------|---------|---------|-------------------
All files          |   91.36 |    87.04 |   81.17 |   91.36 |
 src               |   74.89 |    78.04 |      25 |   74.89 |
  App.tsx          |   74.89 |    78.04 |      25 |   74.89 | ...10-215,242-244
```

```
 src/api           |    98.1 |    89.36 |     100 |    98.1 |
  client.ts        |    98.1 |    89.36 |     100 |    98.1 | 174-176
 src/components    |   92.42 |    85.88 |   68.18 |   92.42 |
  ...ileResult.tsx |     100 |      100 |     100 |     100 |
  IngestPanel.tsx  |   95.48 |     90.9 |   66.66 |   95.48 | 40-42,44-46
  ...eSwitcher.tsx |     100 |    83.33 |     100 |     100 | 27
  QueryBar.tsx     |   86.84 |    76.92 |      40 |   86.84 | ...41-43,68-70,82
  RallyGrid.tsx    |   98.11 |    73.33 |      50 |   98.11 | 42
  ...ionFooter.tsx |      66 |    71.42 |      50 |      66 | 28-50
  StatusChip.tsx   |     100 |      100 |     100 |     100 |
  TapFilters.tsx   |     100 |      100 |     100 |     100 |
 src/hooks         |   92.89 |    76.92 |     100 |   92.89 |
  useIngestJob.ts  |   92.52 |    78.57 |     100 |   92.52 | ...08-110,141-142
  ...ySelection.ts |     100 |    83.33 |     100 |     100 | 9
  useUpload.ts     |    91.8 |    70.58 |     100 |    91.8 | 76-78,82-83
 src/i18n          |     100 |       25 |     100 |     100 |
  index.ts         |     100 |       25 |     100 |     100 | 72-76
 src/lib           |   97.22 |    94.61 |     100 |   97.22 |
  errorText.ts     |     100 |      100 |     100 |     100 |
  errors.ts        |     100 |      100 |     100 |     100 |
  floor.ts         |     100 |      100 |     100 |     100 |
  format.ts        |     100 |      100 |     100 |     100 |
  log.ts           |   81.81 |     87.5 |     100 |   81.81 | 13-14
  query.ts         |   97.07 |    93.45 |     100 |   97.07 | ...95,284,286,288
  selection.ts     |     100 |     92.3 |     100 |     100 | 30
-------------------|---------|----------|---------|---------|-------------------

=============================== Coverage summary ===============================
Statements   : 91.36% ( 1249/1367 )
Branches     : 87.04% ( 356/409 )
Functions    : 81.17% ( 69/85 )
Lines        : 91.36% ( 1249/1367 )
================================================================================
ERROR: Coverage for lines (91.36%) does not meet global threshold (92%)
ERROR: Coverage for statements (91.36%) does not meet global threshold (92%)
```

16. assistant (2026-06-21T19:33:06.606Z)

This is the key finding. With the **same exclude config** but without the two new tests, coverage is 91.36% (lines/stmts), and with the new tests it's 92.61%. So the attributable gain from the new tests is +1.25 points (91.36 ? 92.61), almost entirely from SelectionFooter (66% ? 100%, +17 covered lines).

The "90.44 ? 92.61" framing in the task conflates two separate changes:
1. The exclude config (main.tsx/types.ts removal): 90.44 ? 91.36 (the denominator reframe, ~+0.92)
2. The actual new test (SelectionFooter): 91.36 ? 92.61 (real coverage, ~+1.25)

Let me confirm the 90.44 baseline by checking what main.tsx and types.ts contribute. Let me restore the tests and check the no-exclude baseline, plus examine QueryBar and the i18n fallback uncovered lines.

17. user (2026-06-21T19:33:08.924Z)

```
restored both tests
 M ../.github/workflows/test.yml
 M package-lock.json
 M package.json
 M vite.config.ts
?? src/components/SelectionFooter.test.tsx
?? src/i18n/catalog-invariants.test.ts
```

18. assistant (2026-06-21T19:33:12.319Z)

Good, restored. Now let me check the catalog-invariants test value ? does it add coverage, or is it purely a data-assertion test (no source-line coverage contribution)? And examine QueryBar's uncovered functions and the i18n fallback. Let me look at the lines flagged.

19. user (2026-06-21T19:33:12.900Z)

```
1       // The fast-entry query bar (PER_RALLY_SELECTION.md P3). A deterministic, OFFLINE parser
turns
2       // plain text ("top 15 longest", "over 10s") into a selection over the SAME shared set
the
3       // checkboxes use. It is honest by construction: a live "parsed-pill" shows exactly how
the
4       // text was understood, and concepts with no data behind them (shot-type / player /
score) are
5       // greyed out as "not available yet" ? never silently faked.
6       import { useMemo, useState } from "react";
7       import type { FormEvent } from "react";
8       import { useTranslation, Trans } from "react-i18next";
9       import { parseQuery, summarize } from "../lib/query";
10      import type { ParsedQuery } from "../lib/query";
11
12      interface Props {
13        /** Apply a parsed query to the rally set (App runs runQuery + writes the shared
selection).
14         * `source` distinguishes a typed-and-submitted query from an example-chip tap (for
the trace). */
15        onApply: (q: ParsedQuery, source: "typed" | "example") => void;
16      }
17
18      // Each example is a real, parseable query ? tapping one fills the box AND applies it.
Led by rally
19      // LENGTH (the trusted axis); shot-count filtering exists but is experimental, so it's
not featured
20      // here (a test pins that every example parses to a recognized, fully-available query).
21      export const EXAMPLES = ["over 10s", "top 15 longest", "shortest first", "first 10"];
22
23      // Parsed-pill chips: legible on a phone (text-sm, not text-xs) ? A4.
24      const CHIP = "rounded-full px-3 py-1 text-sm font-medium";
25
26      export function QueryBar({ onApply }: Props) {
27        const { t, i18n } = useTranslation();
28        // The example chips are ENGLISH query syntax (the parser is English-only). For a non-
English user
29        // they're an untappable wall ? hide them and let the localized TapFilters (A9) own
the tap path.
30        const isEnglish = (i18n.resolvedLanguage ?? i18n.language ?? "en").split("-")[0] ===
"en";
31        const [value, setValue] = useState("");
32        const parsed = useMemo(() => parseQuery(value), [value]);
33        const clauses = summarize(parsed);
34        const typed = value.trim().length > 0;
35
36        const applyText = (text: string) => {
37          setValue(text);
38          onApply(parseQuery(text), "example");
39        };
40        const submit = (e: FormEvent) => {
41          e.preventDefault();
42          if (parsed.recognized) onApply(parsed, "typed");
43        };
44
45        return (
46          <div>
47            <form onSubmit={submit} className="flex flex-wrap items-center gap-2">
48              <input
```

```
49                type="text"
50                value={value}
51                onChange={(e) => setValue(e.target.value)}
52                placeholder={t("query.placeholder")}
53                aria-label={t("query.ariaLabel")}
54                className="min-h-11 min-w-[16rem] flex-1 rounded-xl border border-black/15 bg-
white px-4 py-2 text-sm text-ink outline-none focus:border-green"
55              />
56              <button
57                type="submit"
58                disabled={!parsed.recognized}
59                className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
60              >
61                {t("query.apply")}
62              </button>
63            </form>
64
65            {/* Live interpretation ? transparent + hand-correctable (no hidden NL guessing).
*/}
66            <div aria-live="polite" data-testid="query-pill" className="mt-2 flex flex-wrap
items-center gap-1.5">
67              {clauses.map((c) => (
68                <span key={c} className={CHIP + " bg-green/15 text-green"}>
69                  {c}
70                </span>
71              ))}
72              {parsed.unavailable.map((u) => (
73                <span
74                  key={u.token}
75                  title={t("query.notAvailableTitle", { token: u.token, kind: u.kind })}
76                  className={CHIP + " bg-black/5 text-ink/45"}
77                >
78                  <span className="line-through">{u.token}</span> · {t("query.notAvailable")}
79                </span>
80              ))}
81              {typed && !parsed.recognized && parsed.unavailable.length === 0 && (
82                <span className="text-xs text-ink/50">{t("query.unreadable")}</span>
83              )}
84              {!typed && <span className="text-xs text-ink/50">{t("query.hint")}</span>}
85            </div>
86
87            {/* Surface the "not available yet" reason as VISIBLE text (the per-chip title is
hover-only,
88                so it never appears on a phone) ? A4. */}
89            {parsed.unavailable.length > 0 && (
90              <p className="mt-1.5 text-xs text-ink/50">{t("query.notAvailableHelp")}</p>
91            )}
92
93            {isEnglish && (
94              <div className="mt-3 flex flex-wrap items-center gap-2 text-sm">
95                <span className="text-ink/40">{t("query.try")}</span>
96                {EXAMPLES.map((ex) => (
97                  <button
98                    key={ex}
99                    type="button"
100                   onClick={() => applyText(ex)}
101                   className="inline-flex min-h-11 items-center rounded-full border border-
black/15 px-4 text-sm hover:border-green"
102                 >
103                   {ex}
104                 </button>
105               ))}
106             </div>
107           )}
```

```
108
109           <p className="mt-2 text-xs text-ink/40">
110             <Trans i18nKey="query.support" components={{ b: <strong className="text-ink/60"
/>, x: <span className="text-amber-700" /> }} />
111           </p>
112         </div>
113       );
114     }
115
```

20. assistant (2026-06-21T19:33:18.168Z)

QueryBar's uncovered: funcs 40% (3 of 5 funcs uncovered), lines 86.84 (lines 37-43 = `applyText`/`submit`, 68-70/82 = pill rendering). There IS a `QueryBar.examples.test.ts` but it only tests `EXAMPLES` parse via the `lib/query` parser ? it never renders the component. So `applyText` (example-chip tap) and `submit` (form submit) are untested. This is a cheap, high-value render-test win directly analogous to what the PR did for SelectionFooter.

Let me confirm QueryBar.examples.test.ts doesn't render, and check the i18n fallback lines 72-76.

21. user (2026-06-21T19:33:18.908Z)

```
1       import { describe, it, expect } from "vitest";
2       import { EXAMPLES } from "./QueryBar";
3       import { parseQuery } from "../lib/query";
4
5       // The example chips apply on tap, so a broken example would silently do nothing (or
show a
6       // "couldn't read that"). Pin that EVERY shipped example is a recognized, fully-
available query
7       // (no unsupported tokens) ? a regression guard for the duration-led example set.
8       describe("QueryBar examples", () => {
9         it("are all recognized, actionable, and fully available (no experimental/unsupported
tokens)", () => {
10          for (const ex of EXAMPLES) {
11            const q = parseQuery(ex);
12            expect(q.recognized, ex).toBe(true);
13            expect(q.unavailable, ex).toEqual([]);
14            // every example must actually constrain or order the set
15            expect(q.filters.length > 0 || q.sort != null || q.limit != null, ex).toBe(true);
16          }
17        });
18
19        it("lead with rally length, not the experimental shot-count axis", () => {
20          const shotExamples = EXAMPLES.filter((ex) => parseQuery(ex).filters.some((f) =>
f.field === "shot_count"));
21          expect(shotExamples).toEqual([]);
22        });
23      });
24
```

22. assistant (2026-06-21T19:33:29.147Z)

Confirmed ? `QueryBar.examples.test.ts` is a pure `lib/query` parser test (imports `EXAMPLES`, never renders `<QueryBar/>`). So the component's own `applyText`/`submit` functions and pill-rendering branches are entirely uncovered. A render test (~30 lines, same pattern as SelectionFooter) would push QueryBar funcs 40%?~80-100% and lines 86.84?~98%, lifting the whole `functions` metric (currently the tightest at 82.35% vs 80 floor).

Now the i18n fallback (lines 72-76, branch 25%). Line 72 is `(i18n.resolvedLanguage || lng || DEFAULT_LOCALE).split("-")[0]` ? the branch report at 25% means only one of the `||` arms is exercised. This is the `syncDocumentLang` fallback chain. It is genuinely cheap-ish but lower-value: the fallback arms only fire in degenerate states. Let me check whether the catalog-

invariants test contributes any source coverage (it's a data test over JSON, so likely zero source lines) and verify the threshold headroom.

23. user (2026-06-21T19:33:31.971Z)

```
=== threshold headroom check ===
lines/stmts: 92.61 vs floor 92  -> +0.61 slack
branches: 87.74 vs floor 87 -> +0.74 slack
functions: 82.35 vs floor 80 -> +2.35 slack

=== does catalog-invariants cover any SOURCE lines? it imports only node:fs + JSON, no src/
module ===
9:import { describe, it, expect } from "vitest";
10:import { readFileSync, readdirSync } from "node:fs";
11:import { join } from "node:path";
13:const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");
```

24. assistant (2026-06-21T19:33:38.469Z)

Confirmed: `catalog-invariants.test.ts` reads JSON files directly via `node:fs` and imports no `src/` module ? so it contributes **zero** source-line coverage. It's a pure data/regression guard (valuable for locking #77 rules, but it does not move the coverage needle at all). The entire +1.25 attributable coverage gain comes from `SelectionFooter.test.tsx`.

Now let me verify the threshold-comment honesty claim. The vite.config comment says "Floor set just below current (stmts/lines 92.6, branches 87.7, funcs 82.4)". Those match the WITH-tests numbers. But the comment also says functions is "kept looser ... costly to exercise" ? let me check if the functions floor (80) is actually the binding constraint or just slack. Functions has +2.35 slack, the most of any metric ? so a function-coverage regression has the most room to slip silently. Let me also double-check the "90.44" baseline figure by computing the no-exclude number to validate the task's framing.

25. user (2026-06-21T19:33:39.852Z)

```
  18 src/main.tsx
  91 src/api/types.ts
 109 total
--- these are the excluded files; count their non-trivial lines to estimate denominator shift
---
```

26. assistant (2026-06-21T19:33:55.572Z)

I have enough to assess the "90.44" framing. The excludes (main.tsx 18 lines, types.ts 91 lines of type-only declarations) are legitimate non-logic exclusions ? v8 counts types.ts as uncovered runtime lines otherwise. So excluding them is defensible, not gaming. But it does mean ~0.9 of the headline "90.44?92.61" gain is denominator reframing, and ~1.25 is real new coverage (SelectionFooter).

Let me do one final check: confirm the new SelectionFooter test actually drove SelectionFooter to 100% (not some other test), and that the i18n fallback branch (lines 72-76) is the syncDocumentLang one. I've verified both from the before/after tables (SelectionFooter 66%?100%; i18n index branch stays 25% in both runs, so untouched by this PR). I now have everything needed.

Summary of verified facts:
- **Attributable real coverage**: removing the two new tests (same exclude config) drops to **91.36%**; with them, **92.61%**. So the genuine new-test gain is **+1.25 pts, entirely from SelectionFooter** (66?100%, branches 71?100%, funcs 50?100%).
- **catalog-invariants.test.ts adds ZERO source coverage** ? pure JSON/fs data test, imports no src module. Valuable as a #77 regression lock, but contributes nothing to the coverage metric.
- **The "90.44?92.61" headline** blends denominator reframing (excluding main.tsx + types.ty, ~+0.9) with real coverage (~+1.25). The excludes are defensible (type-only / bootstrap), but the headline overstates "added meaningful test coverage."
- **Cheap high-value win left on table**: `QueryBar.tsx` funcs 40%, lines 86.84% ?

`applyText`/`submit` and pill branches untested because the only QueryBar test is a parser-only `lib/query` test that never renders. A render test mirroring SelectionFooter's pattern would lift funcs (the tightest metric, 82.35 vs 80 floor) materially.
- **i18n/index.ts branch 25%** (syncDocumentLang `||` fallback, lines 72-76) is a lower-value, slightly awkward-to-test win (degenerate fallback arms) ? fine as a follow-up.
- **Threshold honesty**: floor numbers in the comment match real WITH-tests numbers. Functions floor (80) has the most slack (2.35) ? the loosest ratchet ? which the comment acknowledges but is where a regression could most easily hide.

27. user (2026-06-21T19:34:25.809Z)

Structured output provided successfully

28. assistant (2026-06-21T19:34:35.184Z)

I've completed the completeness review. Summary of what I verified and found:

Verified locally (by removing the two new tests and re-running coverage with the same exclude config):
- True attributable gain: **91.36% ? 92.61% = +1.25 pts, entirely from SelectionFooter.test.tsx** (66?100%).
- The other ~0.9 of the "90.44?92.61" headline is the denominator reframe (excluding main.tsx + types.ts), which is defensible but should not be sold as new test coverage.
- **catalog-invariants.test.ts adds zero source-line coverage** (pure node:fs JSON data test) ? valuable as a #77 lock, but it does not move the metric.

Single most valuable missing addition (major): a `QueryBar.tsx` render test. It's at funcs 40% / lines 86.84% because the only QueryBar test is a parser-only test that never renders the component, leaving `applyText`/`submit` and the pill branches untested. Functions is the tightest global metric (82.35 vs 80 floor), so this is the cheapest real win left and directly in scope.

Honesty findings: the threshold comment numbers match the real with-tests figures (good), but the functions floor has the most slack (+2.35) and is where a regression could slip; and the PR narrative overstates the coverage delta by blending exclude-reframing with real coverage.

Verdict: **ship-with-fixes** ? the tooling + floor + SelectionFooter test are solid and the SelectionFooter test is genuinely strong; the QueryBar render test and the narrative honesty fixes are the worthwhile gaps.